# CEG5101 Modern Computer Networking

**Graduate Assistants: Ms. Poorvi Joshi (e1144005@u.nus.edu), Ms. Theepana Govintharajah (theepana@u.nus.edu), Ms. Le Minh Chau (le_minhchau@u.nus.edu)**
**Prepared by: Ms. Poorvi Joshi**
**Lecturer: Assoc. Prof. Mohan Gurusamy**
**ECE Department, CDE., National University of Singapore**

---

**Assessment Criteria:**

- **Conceptual Understanding** of SDN routing, network resilience, anomaly detection, and mitigation.
- **Logical Design and Reasoning** in applying the concepts to develop the required SDN solutions.
- **Analysis and Interpretation** of experimental results, graphs, and system performance.

---

## CA2: Laboratory Assignment

Anomaly aware Routing and Blocking Controller

**Objective** **(MM: 30)**

Develop a controller that monitors network traffic, detects abnormally high traffic (like a DDoS attack), blocks the corresponding source IP, and dynamically reroutes legitimate traffic from other sources through better available links.

Use the topology file ceg_mesh_topo.py provided in the ceg_code folder.

**Tasks and Requirements**

**1. Topology in MiniEdit** **(5 marks)**

- Use the provided topology file ceg_mesh_topo.py.
- Create and run the topology in MiniEdit.

**2. Traffic Monitoring, Anomaly Detection and Source Blocking** **(5 marks)**

- Monitor the traffic rate of active flows and maintain an Exponential Moving Average (EMA)-based traffic baseline. When the traffic rate from a source exceeds the defined anomaly threshold, flag the source as anomalous.
- When an anomalous source is detected, install a firewall rule to block its source IP. Verify that the anomalous traffic is no longer forwarded through the network.
- Simulate normal traffic using *ping* to calculate the baselines.
- Simulate attack traffic using *hping3 -icmp --flood*
- Refer to anomaly_detection.py for anomaly detecting and blocking implementation

**3. Dynamic Link-Cost Update** **(10 marks)**

- For each switch-to-switch link, calculate a dynamic routing cost based on its current traffic load and detected anomaly status.
- Increase the cost of congested or anomaly-affected links so that they become less preferable for subsequent routing decisions.
- Refer to Link-Cost in Reference section C to start with

**4. Rerouting of Legitimate Traffic** **(10 marks)**

- Establish a legitimate **h1→h6** flow before introducing any anomaly and record its initial path, link costs, packet loss, and throughput.

- Introduce high-rate traffic successively on the following links:
    a. **s2–s3** using traffic from **h2→h3**
    b. **s2–s6** using traffic from **h2→h6**
    c. **s5–s6** using traffic from **h5→h6**
- For each case:
    a. Observe the change in traffic load and corresponding dynamic link cost.
    b. Verify that the anomalous source is detected and blocked.
    c. Determine whether the **h1→h6** flow is rerouted through an alternative lower-cost path.
    d. Record the **h1→h6 path before and during the anomaly**, together with packet loss and throughput.
    e. Verify that the legitimate **h1→h6** flow remains connected despite the anomaly.
    f. After the anomalous traffic stops, verify that the affected link costs and the **h1→h6** route recover appropriately.
- Refer to resilient_controller.py for the routing implementation.

*The controller should block the anomalous source while preserving connectivity for legitimate traffic through available alternative paths.*

**5. Performance Evaluation:**

1. Include a screenshot of the topology in the report.
2. Evaluate the integrated controller before, during, and after the anomaly.
3. Plot
    a. Dynamic Link Cost (normalized cost) vs. Time
    b. Packet Loss (%) vs. Time
    c. Throughput (Mbps) vs. Time.
4. Explain the changes in routing cost, path selection, packet loss, and throughput during anomaly-aware rerouting.

# Reference

## A. Code - resilient_controller.py

**Code Working:**

- Discovers the network topology by detecting switches, links, and host-facing ports, and builds a network graph.
- Assigns link costs using predefined static costs and uses a default cost when no cost is specified.
- Learns host MAC addresses and uses Dijkstra's shortest-path algorithm to determine the lowest-cost route between switches.
- Installs OpenFlow forwarding rules for known destinations and uses an MST-based flooding mechanism when the destination is unknown.
- Handles link failures by removing the failed link, recomputing the MST, clearing existing flow tables, and allowing new paths to be calculated

**Limitations:**

- **Static routing costs:** Link costs do not change with congestion or traffic load, so it does not perform traffic-aware load balancing.

## B. Code - anomaly_detection.py

**Code Working:**

a. Monitors network traffic statistics by periodically requesting flow statistics from connected switches and calculating packet-count changes for each source IP.
b. Maintains a traffic baseline for each source IP using an exponential moving average with alpha = 0.3.
c. Detects traffic anomalies when the current packet count exceeds 3× the baseline for a source IP.
d. Blocks anomalous source IPs by installing a high-priority OpenFlow drop rule for that IPv4 source across all connected switches.
e. Maintains network state and forwarding information, including blocked IPs, traffic statistics, MAC-to-port mappings, detected events, and basic packet forwarding/flooding.

**Limitations:**

a. **The anomaly threshold is fixed:** current > 3 × baseline, so the detector does not adapt the threshold according to different traffic patterns or traffic types.
b. **No dynamic rerouting/load balancing:** After blocking an anomalous source, the code does not update link costs or calculate an alternative path for legitimate traffic.

### C. Link-Cost

For each network link $(i, j)$, students can calculate a dynamic link cost as

$$C_{ij}(t) = C_{ij}^{0} + \alpha \frac{R_{ij}(t)}{R_{ij}^{\max}} + \beta A_{ij}(t), \tag{1}$$

where:

- $C_{ij}^{0}$ is the original link cost from the provided controller.
- $R_{ij}(t)$ is the current traffic rate on the link.
- $R_{ij}^{\max}$ is the maximum or reference traffic rate of the link.
- $A_{ij}(t)$ indicates whether abnormal traffic is detected:

$$A_{ij}(t) = \begin{cases} 1, & \text{if abnormal traffic is detected,} \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

- $\alpha$ controls the effect of traffic load on the link cost.
- $\beta$ controls the effect of detected anomalies on the link cost.

A higher link cost makes the link less likely to be selected by Dijkstra's algorithm. Students may modify this equation or propose an alternative approach, provided that traffic conditions and detected anomalies are considered when making routing decisions.